

Set Clock Cycle

1. 0xBACAD1E0 is a 32 bit value, addi can only add/load up to 12 bits, so we need lui instruction.

So for loading 0xBACAD1E0 into x22:

lui x22, BACAD

addi x22, x22, 1E0

*N.B: Don't be confused by the word "load", it does not necessarily mean using the load instruction, load instructions are only used when we are transferring data **from memory to register**. Here we are asked to **load into register x22**, not from memory.*

2. Immediate value calculation:

The bne instruction jumps two instructions backward, so it jumps $(2 * 4) = 8$ locations in memory.

So the immediate value is 8

8 in 12 bit binary:

000000001000

After discarding the LSB and sign extending 1 bit we get:

00000000100

Now, as it is a backward jump, we need to convert it to its negative representation(2's complement).

Reverting(1's complement):

00000000100 -> 11111111011

Adding 1(2's complement):

11111111011 + 1 = 11111111100

Final immediate value is: 11111111100

bne x5, x0, loop

Now, converting to SB type machine code:

1	11111	00000	00101	XXX	1100	1	XXXXXXX
Imm [12]	Imm [10:5]	rs2	rs1	func3	Imm [4:1]	Imm [11]	Opcode

Machine code: 11111110000000101XXX11001XXXXXXX

3. RISC-V Assembly:

```
addi x21, x0, 2
addi x22, x0, 3
addi x23, x0, 4
addi x10, x21, 0
addi x11, x22, 0
addi x12, x23, 0
jal x1, kajKormo
sw x10, 48(x20)
lw x5, 48(x20)
slli x6, x5, 3
add x6, x5, x6
sw x6, 52(x20)
beq x0, x0, exit
```

```
kajKormo:
srl x5, x12, x11
or x10, x5, x12
jalr x0, 0(x1)
```

```
exit:
```

Set Double Word

1. 0xABACA1F0 is a 32 bit value, addi can only add/load up to 12 bits, so we need lui instruction.

So for loading 0xABACA1F0 into x22:

```
lui x22, ABACA
```

```
addi x22, x22, 1F0
```

*N.B: Don't be confused by the word "load", it does not necessarily mean using the load instruction, load instructions are only used when we are transferring data **from memory to register**. Here we are asked to **load into register x22**, not from memory.*

2. Immediate value calculation:

The bne instruction jumps two instructions backward, so it jumps $(2 * 4) = 8$ locations in memory.

So the immediate value is 8

8 in 12 bit binary:

```
000000001000
```

After discarding the LSB and sign extending 1 bit we get:

```
000000000100
```

Now, as it is a backward jump, we need to convert it to its negative representation(2's complement).

Reverting(1's complement):

```
000000000100 -> 111111111011
```

Adding 1(2's complement):

```
111111111011 + 1 = 111111111100
```

Final immediate value is: 111111111100

```
bne x7, x0, loop
```

Now, converting to SB type machine code:

1	111111	00000	00111	XXX	1100	1	XXXXXXX
Imm [12]	Imm [10:5]	rs2	rs1	func3	Imm [4:1]	Imm [11]	Opcode

Machine code: 11111110000000111XXX11001XXXXXXX

3. RISC-V Assembly:

```
addi x19, x0, 2
addi x20, x0, 3
addi x21, x0, 4
addi x10, x19, 0
addi x11, x20, 0
addi x12, x21, 0
jal x1, kamKaj
sw x10, 48(x18)
lw x5, 48(x20)
slli x6, x5, 3
sub x6, x5, x6
sw x6, 52(x20)
beq x0, x0, exit
```

```
kamKaj:
srl x5, x12, x11
or x10, x5, x12
jalr x0, 0(x1)
```

```
exit:
```